Tail Waste Fragmentation of KV Cache Using vAttention

Group 3

Joshua Frank, Londy Jean, Beethoven Marhone, Kyle Merritt

Why Use vAttention?

- vAttention reported up to 1.97x faster token generation than PagedAttention variants
- But without the custom driver, GPU paging uses 2 MB pages, which can create tail waste
- Custom vAttention driver reduces page size to 64 KB, which lowers that waste
- The tradeoff is deployment complexity

Research Questions

- Do we still need the vAttention custom driver that reduces page size from 2 MB to 64 KB, even at long context lengths?

- How can a user know in advance when 2 MB pages will cause significant tail waste for a given model and target context range?

Terminology

- **Fragmentation**- wasted or unusable memory due to allocation patterns
  - External – Free memory split into unsuable spaces
  - Internal - Unused space inside allocated memory

- **LLM Inference** – Using a trained model to generate output tokens. Cycles through taking a previous token, running them through a transformer, compute next token probabilities
- **KVCache** – Stores immediate Key(K) and Value tensor from attention for each token layer allowing reuse of past computation

Terminology Continued

- **Context Length** – The total number of tokens processed so far (input + generated output) which determines how the large the KV cache becomes
- **Page/Block** - A fixed- size unit of memory allocation used in paging systems; enables flexible allocation but can introduce internal fragmentation
- **Advanced Attention Architectures** – LLMs can use different methods for the attention part of their computation

Classical Solutions

- **Contiguous Allocation**
  - Allocates one continuous block of memory per request. Memory must be physically adjacent and sized at allocation time
  - Pro
    - Simple and fast memory access due to no indirection
  - Con
    - Severe external fragmentation when request sizes are unpredictable.
- **Memory Compaction**
  - Moves allocated memory blocks to merge scattered free space into once contiguous region
  - Pro
    - Eliminates external fragmentation and restores large contiguous memory blocks
  - Con
    - High overhead due to memory movement and pointer updates; not suitable for real time systems

Classical Solutions Continued

- **Slab / segregated-size allocator**
  - Allocates memory from pools of fixed size objects, where each slab contains identical sized blocks
  - Pro
    - Reduces external fragmentation and improve allocation speed through object reuse
  - Con
    - Causes internal fragmentation and lacks flexibility for dynamically growing data
- **PagedAttention**
  - Divides KV Cache into fixed-size pages and uses a block table to map logical positions to noncontiguous physical memory
  - Pro
    - Eliminates the need for contiguous allocation and support dynamic KV cache growth
  - Con
    - Introduces kernel complexity and internal fragmentation due to fixed page size

Why Classical Methods Break for KV Cache

- **Dynamic KV growth** – Requests grow unpredictably during the inference
- **Real-time latency constraints –** Memory operations must be fast no expensive relocation
- **Mismatch with classical assumptions** - Traditional systems assume static sizes to allow memory movement
- The overall down fall is that these traditional systems leads to fragmentation, inefficiency, or performance overhead

Vattention

- VAttention addresses fragmentation by introducing a virtual memory abstraction for the KV cache
- Instead of requiring contiguous physical memory it allows the KV cache to grow dynamically using noncontiguous pages, while still appearing contiguous to the model.
- Avoids over allocation and eliminates the need for expensive memory movement

Attention Architectures

- Which less memory for the KV cache according to theory?
- MHA – Good
- GQA – Better
- MLA – Best

Tail Waste Expressions

$$F_{\text{exact}}(C) = 100 \cdot \frac{\left\lceil \frac{C}{T} \right\rceil T - C}{\left\lceil \frac{C}{T} \right\rceil T}$$

$$F_{\text{worst}}(C) \approx 100 \cdot \frac{T}{C + T}$$

Variable meanings

- C: context length in tokens for the request.
- T: number of cached tokens that fit in one physical page.
- $\left\lceil \frac{C}{T} \right\rceil$: number of pages mapped for context length C.
- $\left\lceil \frac{C}{T} \right\rceil T$: number of tokens that could be held by all mapped blocks
- $\left\lceil \frac{C}{T} \right\rceil T - C$: unused token capacity in the final mapped page, i.e., tail waste.
- $F_{\text{exact}}(C)$: exact tail-waste percentage at context length C.
- $F_{\text{worst}}(C)$: smooth worst-case upper envelope of the fragmentation peaks.

Token Per Page Expressions

$$T_{\text{dense}} = \left\lfloor \frac{P}{H_{\text{kv,loc}} D b} \right\rfloor$$

$$T_{\text{MLA}} = \left\lfloor \frac{P}{(r_{\text{kv}} + d_{\text{rope}}) b} \right\rfloor$$

Where the terms come from

- P: physical page size chosen by the runtime. In our experiments, this is fixed at 2 MB.
- $H_{\text{kv,loc}}$: local KV-head count, determined by the model architecture and tensor-parallel setup.
- D: head dimension from the trained model configuration.
- b: bytes per stored KV-cache element, determined by the cache datatype such as FP16 or BF16.
- r_{kv}: MLA KV latent rank from the model configuration.
- d_{rope}: MLA RoPE-retained key dimension from the model configuration.

END-TO-END PIPELINE: FRAGMENTATION WORKFLOW

1. SERVER STARTUP

2. RUN REQUEST SWEEP

SERIAL REQUEST SWEEP

REQUEST

Running

CONTROL CONTEXT LENGTH

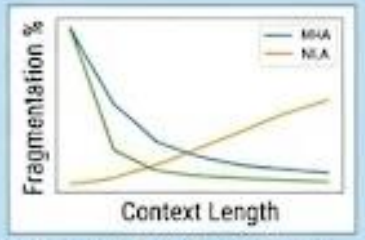
3. SERVER SHUTDOWN

4. SAVE METRICS

CSV

SAVE EXPERIMENT METRICS

5. ANALYZE & PLOT

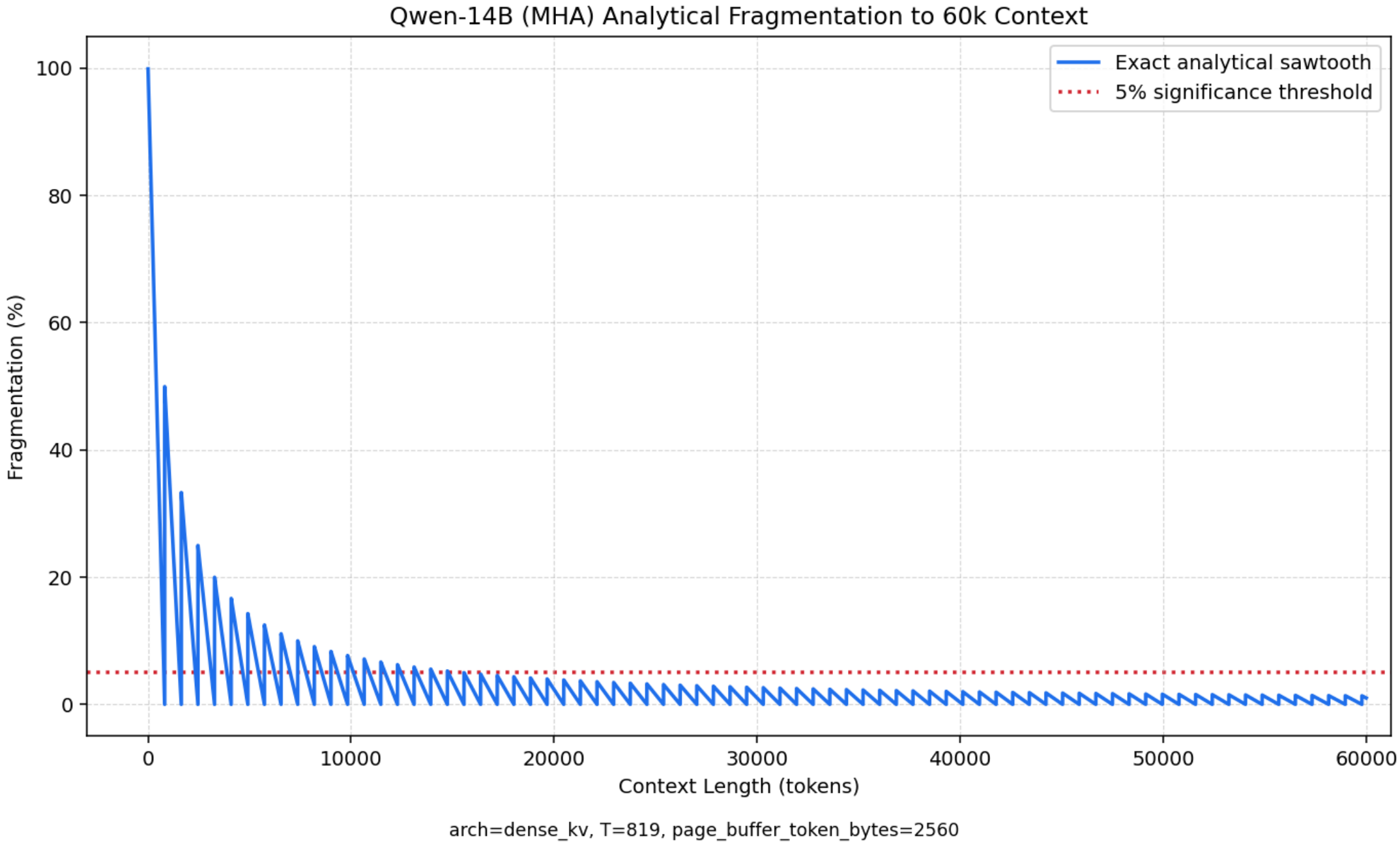


MEASURE FRAGMENTATION & & GENERATE VISUALS

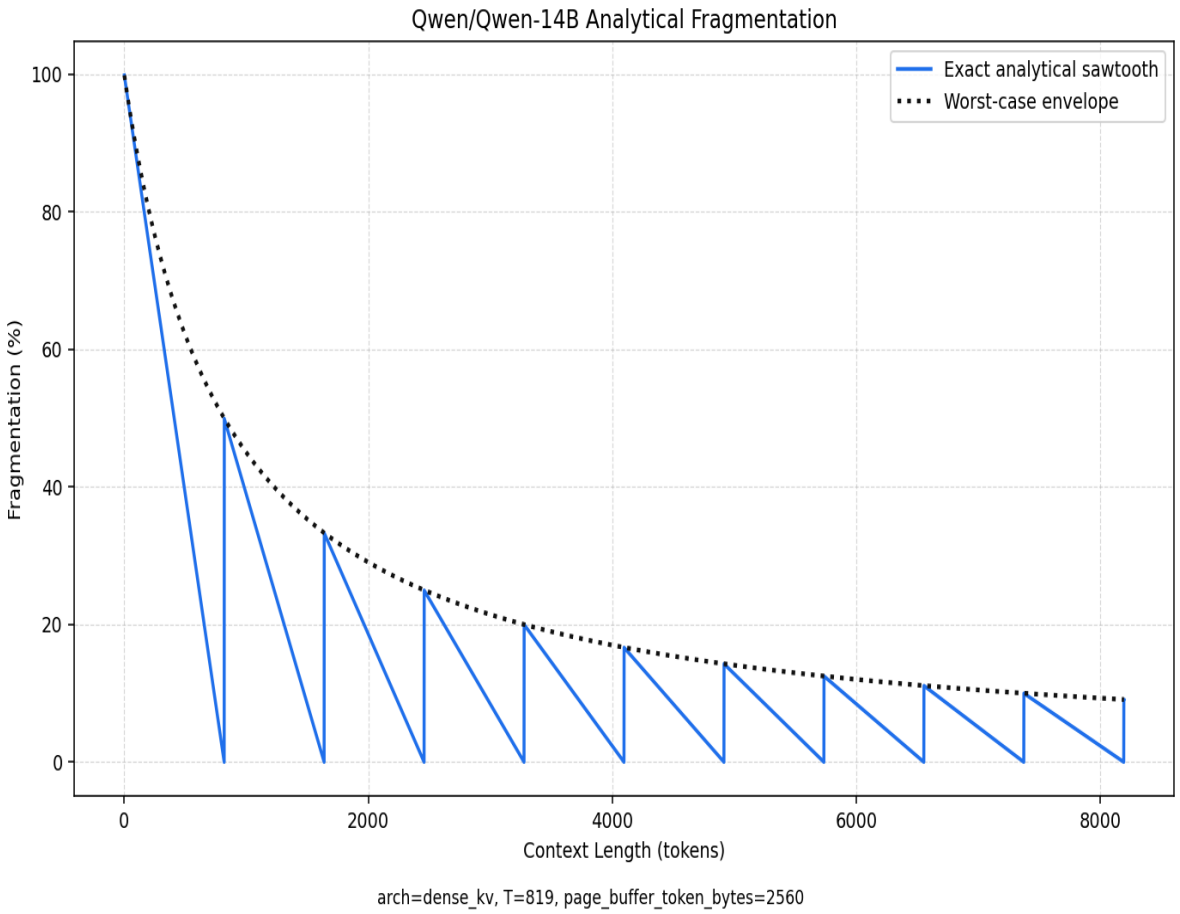
MEASURING FRAGMENTATION ACROSS ATTENTION ARCHITECTURES

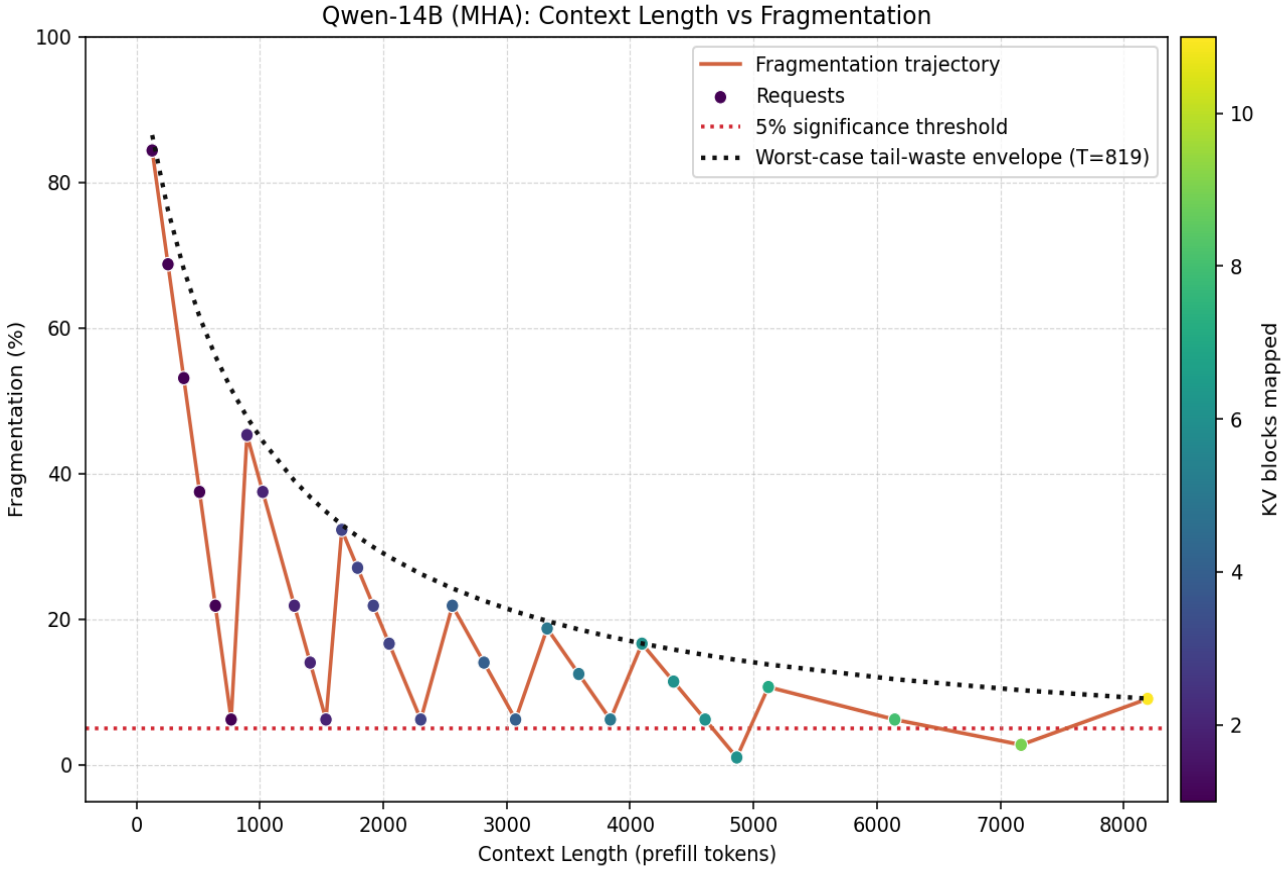
Side Note: Synthetic MLA

- Allows apples-to-apples comparison between GQA and MLA
- We convert Mistral-Nemo-12B to use an MLA-style cache
- Cache allocation behavior is meaningful
- Not a usable language model after conversion

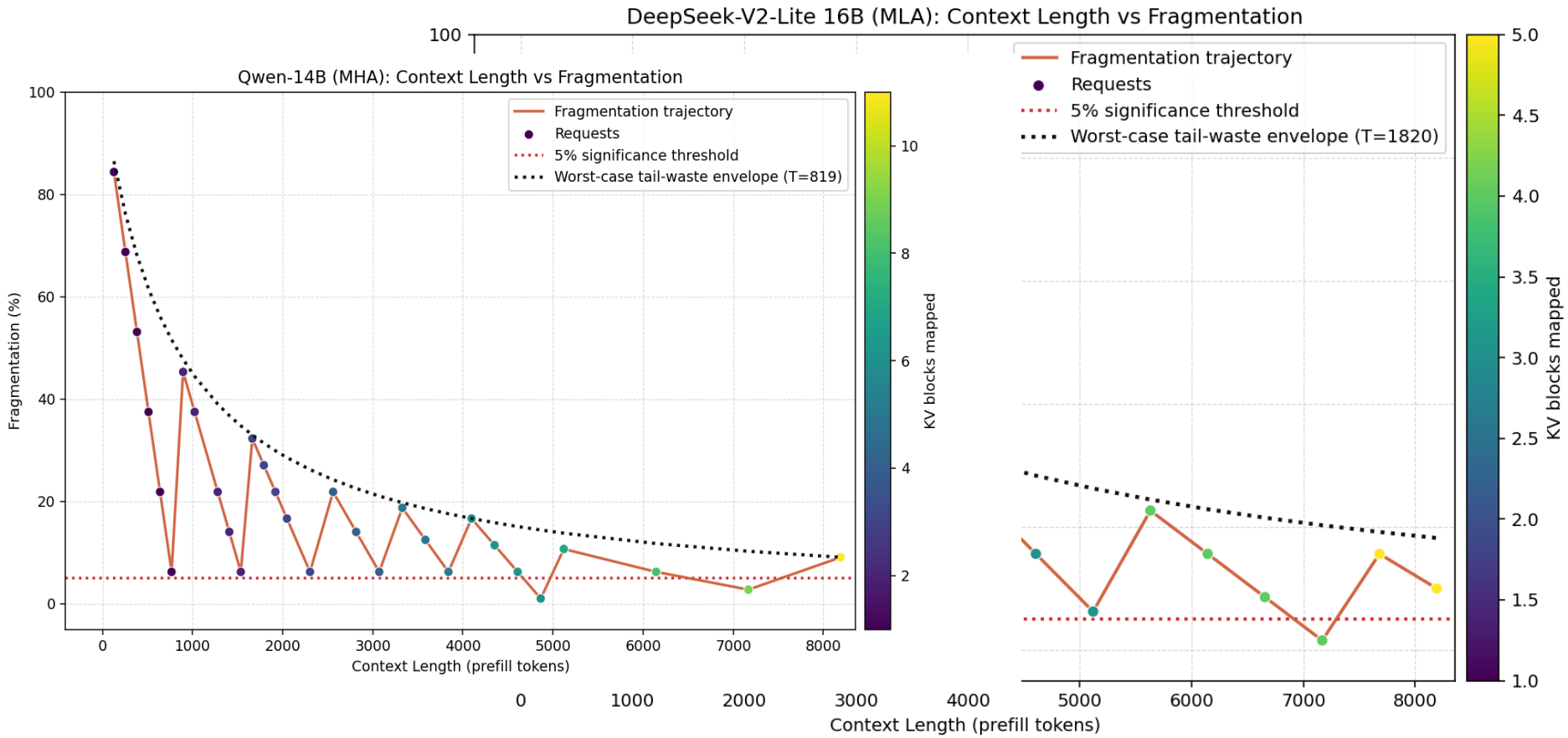

Qwen-14B (MHA) Analytical Fragmentation to 60k Context
Exact analytical sawtooth
5% significance threshold
Fragmentation (%)
100
80
60
40
20
0
0
10000
20000
30000
40000
50000
60000
Context Length (tokens)
arch=dense_kv, T=819, page_buffer_token_bytes=2560

Analytical vs Empirical Fragmentation

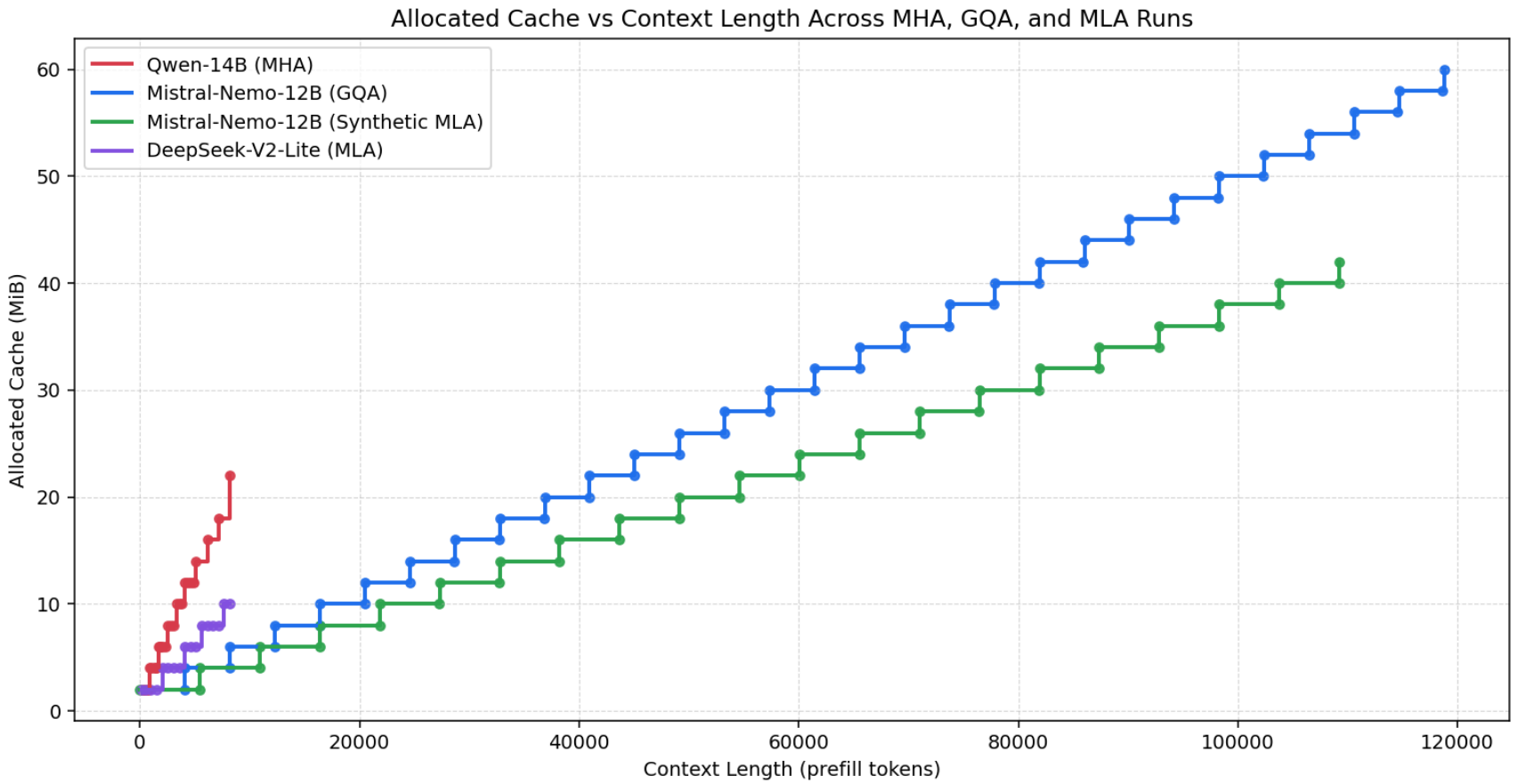




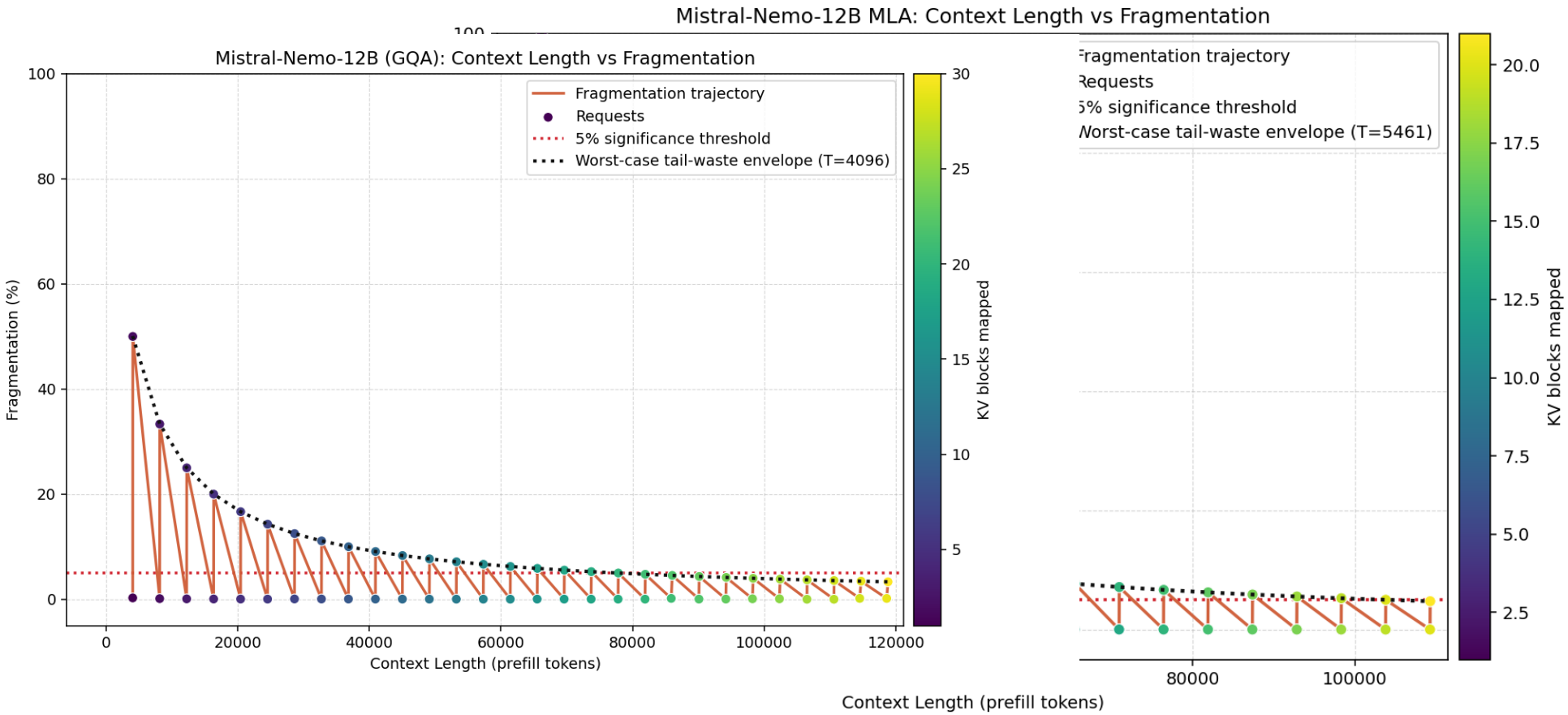
MHA vs MLA

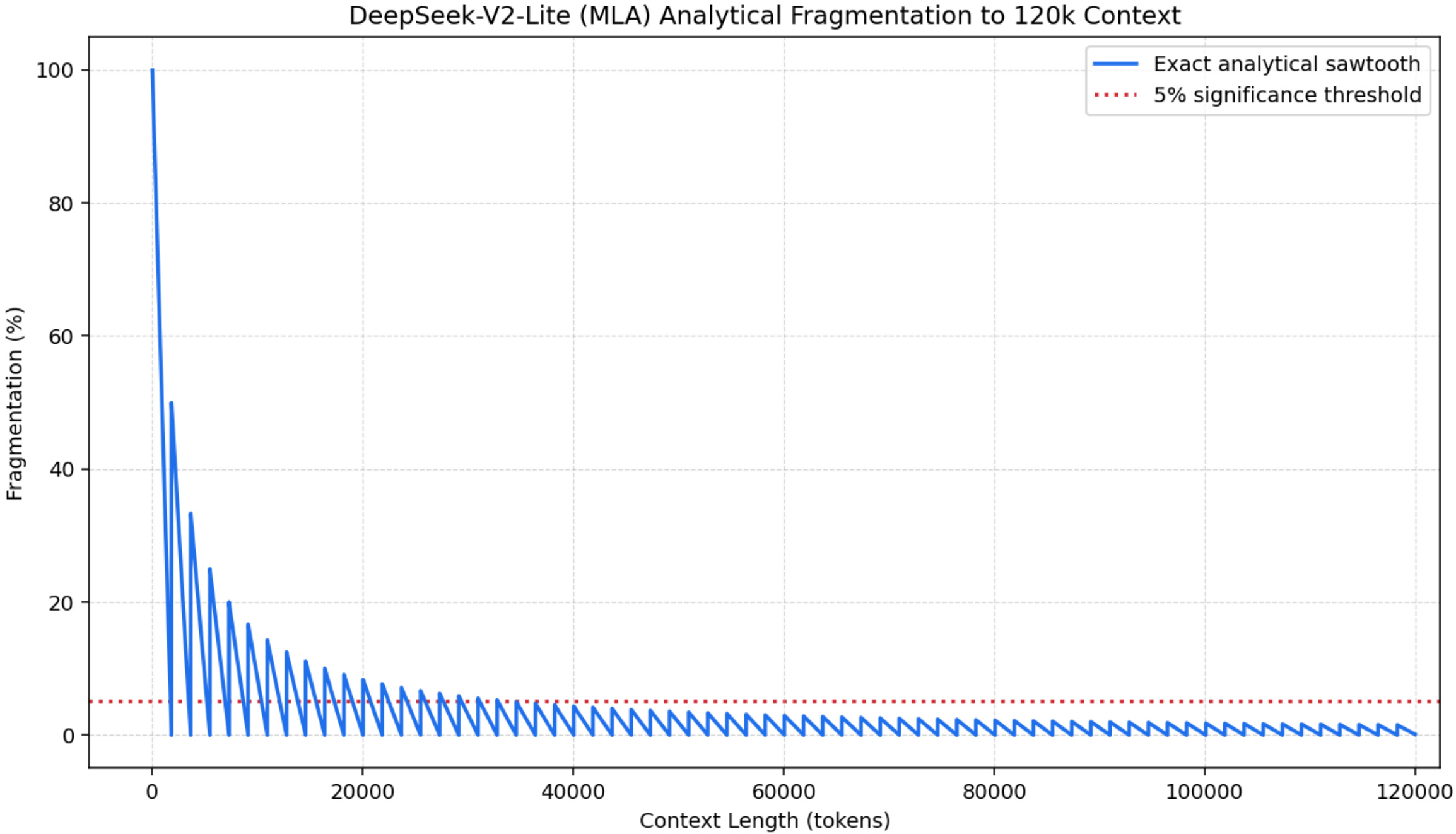

Qwen-14B (MHA): Context Length vs Fragmentation
Fragmentation trajectory
Requests
5% significance threshold
Worst-case tail-waste envelope (T=819)
Fragmentation (%)
Context Length (prefill tokens)
KV blocks mapped
DeepSeek-V2-Lite 16B (MLA): Context Length vs Fragmentation
Fragmentation trajectory
Requests
5% significance threshold
Worst-case tail-waste envelope (T=1820)
Context Length (prefill tokens)
KV blocks mapped

MHA vs GQA vs MLA



GQA vs MLA


Mistral-Nemo-12B (GQA): Context Length vs Fragmentation
Fragmentation trajectory
Requests
5% significance threshold
Worst-case tail-waste envelope (T=4096)
Fragmentation (%)
Context Length (prefill tokens)
KV blocks mapped
Mistral-Nemo-12B MLA: Context Length vs Fragmentation
Fragmentation trajectory
Requests
5% significance threshold
Worst-case tail-waste envelope (T=5461)
Context Length (prefill tokens)
KV blocks mapped


DeepSeek-V2-Lite (MLA) Analytical Fragmentation to 120k Context
Exact analytical sawtooth
5% significance threshold
Fragmentation (%)
100
80
60
40
20
0
0
20000
40000
60000
80000
100000
120000
Context Length (tokens)
arch=mla, T=1820, page_buffer_token_bytes=1152

Conclusion

- Need for the custom driver depends on model architecture and context length.

- Our analytical model predicts fragmentation well and helps guide deployment decisions.